论文精读

Parallelizing Linear Transformers with the Delta Rule over Sequence Length

在序列长度维度上并行化带有Delta规则的线性Transformer

原文作者：Songlin Yang[◇], Bailin Wang[◇], Yu Zhang[†], Yikang Shen[‡], Yoon Kim[◇]

[◇]MIT [†]Soochow University [‡]MIT-IBM Watson AI Lab

发表于 NeurIPS 2024

本文档为该论文的全文逐段精读与解读

编译方式: `latexmk -xelatex deltanet_reading.tex`

Contents

1 摘要 (Abstract) **2**

2 引言 (Introduction) **3**

3 背景 (Background) **4**
3.1 线性Transformer：具有线性注意力的Transformer 4
3.2 DeltaNet：具有Delta更新规则的线性Transformer 8

4 在序列维度上并行化DeltaNet **10**
4.1 内存高效的重参数化 10
4.2 DeltaNet的分块并行形式 12
4.3 DeltaNet Transformer 16
4.4 混合模型 17

5 实验研究 **18**
5.1 合成基准测试 18
5.1.1 MQAR（多查询关联检索） 18
5.1.2 MAD基准 19
5.2 语言建模 19
5.2.1 训练吞吐量 21

6 讨论与相关工作 **22**
6.1 DeltaNet vs. 状态空间模型/线性RNN 22
6.2 局限性与未来工作 23
6.3 相关工作 23

7 结论 **24**

8 致谢 **24**

A 附录 **24**
A.1 超参数设置 24
A.2 合成任务：RegBench 25
A.3 WY表示推导 25
A.4 UT变换推导 26
A.5 伪代码 28

B 参考文献一览表 **29**
B.1 线性递归模型总览 30

1 摘要 (Abstract)

原文翻译

具有线性注意力的Transformer（即线性Transformer）和状态空间模型最近被提出作为softmax注意力Transformer的一种线性时间替代方案。然而，这些模型仍然在性能上不及Transformer，尤其是在需要上下文检索（in-context retrieval）的任务上。虽然线性Transformer的更具表达力的变体——用delta规则替代线性Transformer中的加性更新的DeltaNet [Schlag et al., 2021]——已被发现在关联记忆（associative recall）任务上更有效，但现有的训练此类模型的算法无法在序列长度上并行化，因此在现代硬件上训练效率低下。

本工作描述了一种硬件高效的算法，用于训练带有delta规则的线性Transformer。该算法利用了Householder矩阵乘积的紧凑WY表示 [Bischof & Van Loan, 1985] 来实现内存高效的计算。这种算法使我们能够将DeltaNet扩展到标准的语言建模设置中。我们训练了一个1.3B参数的模型，使用100B个token进行训练，发现它在困惑度（perplexity）和下游任务的零样本性能方面优于最近的线性时间基线模型，如Mamba [Gu & Dao, 2023] 和 GLA [Yang et al., 2023]。我们还实验了两种混合模型，分别将DeltaNet层与(1)每隔一层的滑动窗口注意力层或(2)两层全局注意力层结合，发现这些混合模型优于强大的Transformer基线。

并行DeltaNet层作为FlashLinearAttention库的一部分提供：

https://github.com/fla-org/flash-linear-attention

通俗解释

这篇论文解决的核心问题是：线性Transformer虽然速度快（线性复杂度），但在”检索”能力上不如标准Transformer。有一种叫做DeltaNet的改进模型可以提升检索能力，但它原来的训练算法是逐步处理的，无法利用GPU的并行能力，导致训练很慢。

作者的贡献是发明了一种新的训练算法，让DeltaNet可以在序列长度维度上并行训练。这就像是把一条很长的流水线拆成多段同时处理，大大提高了效率。具体方法借用了数值线性代数中的”WY表示”技巧。

关键成果：训练了1.3B参数的DeltaNet模型，性能超过了Mamba和GLA等同类模型；混合模型（DeltaNet + 注意力）甚至超过了标准Transformer。

关键概念

论文三大贡献：(1) 提出DeltaNet的硬件高效分块并行训练算法（基于WY表示）；(2) 将DeltaNet扩展到1.3B规模语言建模，验证其优于Mamba/GLA；(3) 设计混合架构（DeltaNet + 滑动窗口/全局注意力），超越Transformer++基线。

2 引言 (Introduction)

原文翻译

注意力机制 [Bahdanau et al., 2015; Vaswani et al., 2017] 【Bahdanau et al., 2015 提出了神经机器翻译中的注意力机制；Vaswani et al., 2017 提出了Transformer架构。它们是注意力机制的奠基性工作。】 已被证明是精确序列建模的重要基本组件。注意力在训练时也很高效，因为它包含丰富的矩阵乘法运算，可以利用现代GPU上的高度并行处理能力和专用加速器。然而，注意力的复杂度是序列长度的二次方，因此它是一个根本性的昂贵操作。尽管最近的技术通过硬件感知的中间计算重组使注意力能够扩展到更长序列 [Dao, FlashAttention, 2022; Dao, FlashAttention-2, 2023; Liu, Ring Attention, 2023; Brandon, Striped Attention, 2023] 【这些是FlashAttention系列和Ring/Striped Attention工作，通过硬件感知的算法优化加速了softmax注意力计算。】，这些方法仍然需要存储之前元素的键/值向量，而这个”KV缓存”（其大小线性增长）在长序列下难以管理。

线性注意力Transformer [Katharopoulos et al., 2020] 【Katharopoulos et al., 2020 首次提出了线性注意力，将softmax中的指数核替换为特征映射的点积，使注意力变为线性复杂度。】 将softmax注意力中的指数核替换为（可能经过变换的）键和查询向量的点积。这使得可以将线性注意力表示为具有矩阵值隐状态的线性RNN，从而不需要KV缓存并实现常数内存推理。虽然线性注意力的初始变体在语言建模上通常不如softmax注意力，但带有数据依赖门控因子的门控线性注意力变体最近已被证明可以与强大的Transformer基线竞争 [Yang, GLA, 2023; Qin, HGRN2, 2024; Beck, xLSTM, 2024; Peng, RWKV-6, 2024] 【这些都是近期的门控线性注意力/线性RNN变体，它们通过引入数据依赖的门控机制提升了线性注意力的性能。】。 这些门控线性Transformer，连同时变状态空间模型如Mamba [Gu & Dao, 2023; Dao & Gu, Mamba-2, 2024] 【Mamba是选择性状态空间模型，可以被重参数化为门控线性Transformer。】 （可以重参数化为门控线性Transformer），已被提出作为普通Transformer的潜在替代方案。然而，尽管语言建模性能具有竞争力，这些模型在召回密集任务上已被证明不如Transformer [Arora et al., 2024a, 2024b] 【Arora et al. 的工作系统研究了线性注意力模型在召回任务上的不足。】， 这对许多实际的下游任务很重要（例如，检索增强生成 [Lewis et al., RAG, 2021]）。

为了增强长上下文的关联检索能力，Schlag等人(2021)提出了DeltaNet，一种线性Transformer的变体，它使用类delta规则的更新 [Widrow & Hoff, 1988] 【Widrow-Hoff学习规则（LMS算法）是一种经典的自适应滤波/在线学习算法，基于预测误差（delta）来更新权重。DeltaNet的名字就来源于此。】 来检索和更新与当前键相关联的值向量。DeltaNet在合成任务和小规模语言建模/机器翻译上被发现是有效的。然而，原始工作使用的是不能在序列长度上并行化的顺序算法，因此导致硬件效率低下的训练，目前还不清楚如何将DeltaNet扩展到更大的模型和数据集。

本工作描述了一种硬件高效的DeltaNet训练算法，它在序列长度维度上并行化前向/反向传播。我们将DeltaNet重参数化为一个矩阵值RNN，其递推由广义Householder变换给出。这种重参数化使得可以使用Householder矩阵乘积的紧凑WY表示 [Bischof & Van Loan, 1985] 【WY表示是数值线性代数中用于高效表示Householder矩阵乘积的方法，

避免了显式计算大型矩阵乘积。】，消除了在并行训练期间每个时间步具现化矩阵大小隐状态的需要，否则会导致高I/O成本。内存高效的表示使得可以直接将线性注意力模型的分块并行训练策略 [Hua et al., 2022; Sun et al., RetNet, 2023; Yang et al., GLA, 2023] 扩展到DeltaNet的情况。
我们将DeltaNet扩展到中等规模的语言建模基准（1.3B模型在100B token上训练），发现DeltaNet在语言建模和零样本下游任务性能上优于强大的线性递归模型，如Mamba和GLA。对于上下文检索和学习评估，我们在合成和真实基准上评估DeltaNet，发现它在线性递归基线上表现良好。最后，我们实验了一种混合方法，将DeltaNet层与滑动注意力层或全局注意力层结合，发现这些混合模型可以优于普通Transformer以及纯DeltaNet Transformer。

通俗解释

引言梳理了一条清晰的研究脉络：
问题起点：标准Transformer的注意力机制虽然强大但复杂度是$O(L^2)$，长序列下很慢且KV缓存消耗大量内存。
已有尝试：线性注意力将复杂度降到$O(L)$，但性能不够好，尤其在需要”检索”的任务上。即使加了门控（GLA、Mamba等），检索能力仍然不足。
DeltaNet的优势：用delta规则（类似于”看到新信息时先删除旧信息再写入新信息”）替代简单的加性更新，检索能力更强。类比：普通线性注意力就像只会往笔记本上加内容，越写越满越找不到东西；DeltaNet则会先擦掉相关旧内容再写新的。
DeltaNet的瓶颈：原来的训练算法是严格串行的（第t步必须等第$t-1$步完成），GPU利用率很低。
本文的解决方案：通过巧妙的数学重参数化（WY表示），发现可以把DeltaNet的训练拆成”块”来并行处理，就像把长序列切成小段，每段内部并行计算，段之间传递状态。

注意事项

本文的核心贡献是**算法层面**的——提出了一种新的训练算法，而非新的模型架构。DeltaNet模型本身是Schlag等人(2021)提出的，本文的关键创新在于如何高效地训练它。这一区分很重要，因为很多读者可能误以为DeltaNet是本文的原创模型。

3 背景 (Background)

3.1 线性Transformer：具有线性注意力的Transformer

原文翻译

给定一系列d维输入向量$\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L$，Transformer使用softmax注意力机制来关注整个

过去:

$$\boldsymbol{q}_t,\ \boldsymbol{k}_t,\ \boldsymbol{v}_t = \mathbf{W}_Q\boldsymbol{x}_t, \mathbf{W}_K\boldsymbol{x}_t, \mathbf{W}_V\boldsymbol{x}_t, \qquad \boldsymbol{o}_t = \sum_{i=1}^{t} \frac{\exp(\boldsymbol{k}_i^\top \boldsymbol{q}_t)}{\sum_{j=1}^{t}\exp(\boldsymbol{k}_j^\top \boldsymbol{q}_t)}\boldsymbol{v}_i, \tag{1}$$

其中 $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d\times d}$，$\boldsymbol{q}_t, \boldsymbol{k}_t, \boldsymbol{v}_t, \boldsymbol{o}_t \in \mathbb{R}^d$。（这里为简单起见假设单注意力头。）
线性注意力 [Katharopoulos et al., 2020]【线性注意力的开创性工作，将指数核替换为特征映射点积。】将指数核 $\exp(\boldsymbol{k}_i^\top \boldsymbol{q}_t)$ 替换为点积 $\phi(\boldsymbol{k}_i)^\top\phi(\boldsymbol{q}_t)$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是特征映射。这使得可以重新安排计算，将线性注意力表示为具有矩阵值隐状态的线性RNN:

$$\boldsymbol{o}_t = \sum_{i=1}^{t} \frac{\phi(\boldsymbol{k}_i)^\top\phi(\boldsymbol{q}_t)}{\sum_{j=1}^{t}\phi(\boldsymbol{k}_j)^\top\phi(\boldsymbol{q}_t)}\boldsymbol{v}_i = \frac{\left(\sum_{i=1}^{t}\boldsymbol{v}_i\phi(\boldsymbol{k}_i)^\top\right)\phi(\boldsymbol{q}_t)}{\left(\sum_{j=1}^{t}\phi(\boldsymbol{k}_j)^\top\right)\phi(\boldsymbol{q}_t)} = \frac{\mathbf{S}_t\phi(\boldsymbol{q}_t)}{\boldsymbol{z}_t^\top\phi(\boldsymbol{q}_t)}, \tag{2}$$

其中 $\mathbf{S}_t = \sum_{i=1}^{t}\boldsymbol{v}_i\phi(\boldsymbol{k}_i)^\top \in \mathbb{R}^{d\times n}$，$\boldsymbol{z}_t = \sum_{i=1}^{t}\phi(\boldsymbol{k}_i) \in \mathbb{R}^n$。
如果允许n趋于无穷，线性注意力可以使用与多项式核相关的特征映射来计算指数核的多项式近似作为点积，从而可以任意精确地近似softmax注意力。分母 $\boldsymbol{z}_t^\top\phi(\boldsymbol{q}_t) \in \mathbb{R}$ 可能导致数值不稳定性，在近期工作中被移除。常用恒等映射作为ϕ，得到以下简化的线性Transformer:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t\boldsymbol{k}_t^\top, \qquad \boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t. \tag{3}$$

通俗解释

这段介绍了从标准注意力到线性注意力的推导过程。
标准softmax注意力：对于序列中的第t个位置，计算它与所有过去位置的”关联度”（通过$\exp(\boldsymbol{k}_i^\top\boldsymbol{q}_t)$），然后用归一化后的关联度对值向量$\boldsymbol{v}_i$加权求和。这像是在”查字典”——用查询$\boldsymbol{q}_t$去匹配所有的键$\boldsymbol{k}_i$，找到最相关的值$\boldsymbol{v}_i$。
线性注意力的关键洞察：如果把$\exp(\boldsymbol{k}_i^\top\boldsymbol{q}_t)$替换为$\phi(\boldsymbol{k}_i)^\top\phi(\boldsymbol{q}_t)$（即核函数换成线性的），那么求和的顺序可以交换！原来必须先计算每对(i,t)的相似度（$O(L^2)$），现在可以先把$\boldsymbol{v}_i$和$\phi(\boldsymbol{k}_i)$累积成矩阵$\mathbf{S}_t$（$O(L)$步），每步只需$O(d^2)$。
最简形式：去掉特征映射和归一化后，线性注意力就是：维护一个$d\times d$的”记忆矩阵”$\mathbf{S}_t$，每一步加上新的外积$\boldsymbol{v}_t\boldsymbol{k}_t^\top$，输出就是$\mathbf{S}_t$乘以查询$\boldsymbol{q}_t$。

数学推导详解：线性注意力的结合律

目标：展示如何从注意力公式推导出矩阵递推形式。
公式 (2) 中的关键步骤是利用结合律交换求和顺序。设ϕ为恒等映射，省略归一化:

$$\boldsymbol{o}_t = \sum_{i=1}^{t}(\boldsymbol{k}_i^\top\boldsymbol{q}_t)\boldsymbol{v}_i = \sum_{i=1}^{t}\boldsymbol{v}_i(\boldsymbol{k}_i^\top\boldsymbol{q}_t)$$

注意 $\boldsymbol{v}_i(\boldsymbol{k}_i^\top \boldsymbol{q}_t) = (\boldsymbol{v}_i \boldsymbol{k}_i^\top)\boldsymbol{q}_t$，因为 $\boldsymbol{k}_i^\top \boldsymbol{q}_t$ 是标量。因此：

$$\boldsymbol{o}_t = \left(\sum_{i=1}^{t} \boldsymbol{v}_i \boldsymbol{k}_i^\top\right) \boldsymbol{q}_t = \mathbf{S}_t \boldsymbol{q}_t$$

其中 $\mathbf{S}_t \in \mathbb{R}^{d\times d}$。递推形式为 $\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$。
维度分析：$\boldsymbol{v}_i \in \mathbb{R}^d$（列向量），$\boldsymbol{k}_i^\top \in \mathbb{R}^{1\times d}$（行向量），所以 $\boldsymbol{v}_i \boldsymbol{k}_i^\top \in \mathbb{R}^{d\times d}$（外积），$\mathbf{S}_t \in \mathbb{R}^{d\times d}$，$\mathbf{S}_t \boldsymbol{q}_t \in \mathbb{R}^d$，与输出$\boldsymbol{o}_t$维度匹配。
数值例子：设$d = 2$，$\boldsymbol{k}_1 = [1, 0]^\top$，$\boldsymbol{v}_1 = [3, 4]^\top$，$\boldsymbol{q}_1 = [0.5, 0.5]^\top$。

$$\mathbf{S}_1 = \boldsymbol{v}_1 \boldsymbol{k}_1^\top = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix}, \quad \boldsymbol{o}_1 = \mathbf{S}_1 \boldsymbol{q}_1 = \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix}$$

原文翻译：高效训练

设 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{L\times d}$ 为堆叠的查询、键、值向量，例如 $\mathbf{Q}_i = \boldsymbol{q}_i$。我们可以通过以下方式并行计算输出 $\mathbf{O} \in \mathbb{R}^{L\times d}$：$\mathbf{O} = \left(\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M}_L\right)\mathbf{V}$，其中 $\mathbf{M}_L \in \mathbb{R}^{L\times L}$ 是因果掩码。这种完全”并行形式”和上述”递推形式”有不同的浮点运算量和并行化权衡。并行形式需要$O(L^2 d + Ld^2)$，因此比递推形式$O(Ld^2)$需要更多浮点运算。然而，并行形式在实践中对于中等长度序列通常快得多，因为它可以在$O(1)$步内完成。这种序列级并行性也实现了高GPU占用率。递推形式需要更少的浮点运算，但不能在序列长度上并行化[a]，而且递推中涉及的逐元素操作无法利用专用矩阵乘法加速器（如tensor core）。

[a]理论上可以使用parallel scan来并行化递推形式，这可以在$O(\log L)$步和$O(Ld^2)$浮点运算中完成。然而，这种方法需要为每个时间步具现化二维隐状态，这会产生显著的内存I/O成本，除非状态大小足够小以至于可以在更快的内存中具现化（即如Mamba中那样）。

通俗解释

这段说明了线性注意力的两种计算形式各有优劣：
并行形式（一次性算出所有输出）：像是考试时一次性把所有题目都看了然后一起答——虽然总工作量大（$O(L^2 d)$），但因为所有计算可以同时进行，在GPU上很快。
递推形式（逐步累积）：像是流水线上一个一个处理——总工作量少（$O(Ld^2)$），但必须等前一步完成才能做下一步，GPU大部分时间在闲置。
核心矛盾：浮点运算量 vs 并行度。GPU有大量核心但需要足够的并行任务才能充分利用。

原文翻译：分块并行形式

分块并行形式 [Hua et al., 2022; Sun et al., RetNet, 2023; Yang et al., GLA, 2023] 【这三篇工作分别独立发现了线性注意力的分块并行计算方法，是本文算法的直接基础。】 在并行形式和递推形式之间取得平衡，允许比并行形式更少的浮点运算和比递推形式更多的序列级并行性。具体地，假设查询/键/值向量被分成$\frac{L}{C}$个块，每个块的长度为C。设$\mathbf{Q}_{[t]} \in \mathbb{R}^{C\times d}$为第$t$块的所有查询向量，$\boldsymbol{q}_{[t]}^i = \boldsymbol{q}_{tC+i}$为第$t$块中第$i$个查询向量；键/值块的定义类似。注意$t \in [0, L/C)$，$i \in [1, C]$。状态矩阵也相应重新索引，使得$\mathbf{S}_{[t]}^i = \mathbf{S}_{tC+i}$，

另外定义$\mathbf{S}_{[t]}^0 = \mathbf{S}_{[t-1]}^C$，即一个块的初始状态是前一个块的最终状态。
我们可以得到以下关于第t块中第r个元素的隐状态和输出向量的恒等式：

$$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]}^0 + \sum_{i=1}^{r} \boldsymbol{v}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top}, \qquad \boldsymbol{o}_{[t]}^r = \mathbf{S}_{[t]}^0 \boldsymbol{q}_{[t]}^r + \sum_{i=1}^{r} \boldsymbol{v}_{[t]}^i \left(\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{q}_{[t]}^r\right).$$

通过进一步用并行形式改写块内计算，我们得到：

$$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]} + \mathbf{V}_{[t]}^\top \mathbf{K}_{[t]} \qquad \in \mathbb{R}^{d\times d}, \tag{4}$$

$$\mathbf{O}_{[t]} = \mathbf{Q}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{M}_C\right) \mathbf{V}_{[t]} \qquad \in \mathbb{R}^{C\times d} \tag{5}$$

其中我们令$\mathbf{S}_{[t]} = \mathbf{S}_{[t]}^0$以简化符号。在这种”分块并行形式”下，信息通过$\mathbf{S}_{[t]}$在块之间传播，块内状态$\mathbf{S}_{[t]}^i$（$i \in [1, C]$）不需要被具现化，从而节省内存。
分块并行形式的复杂度为$O(LCd + Ld^2)$，步数（不考虑块级parallel scan）为$O(\frac{L}{C})$。因此，$C = L$恢复完全并行形式，$C = 1$恢复递推形式。分块并行形式允许我们在两种形式之间插值，本质上是在顺序计算步数和序列级并行性之间进行权衡。实践中C设为小常数（通常64或128），允许亚二次训练。

通俗解释

分块并行形式是一个精妙的折中方案。想象你要处理一本有L页的书：
并行形式（$C = L$）：一次性看完所有页面并行处理——工作量大但同时进行。
递推形式（$C = 1$）：逐页阅读，每页都要参考之前的笔记——工作量小但串行。
分块并行形式：把书分成每C页一章，章内并行处理，章间串行传递”读书笔记”$\mathbf{S}_{[t]}$。这样既利用了GPU的并行能力（章内C页同时处理），又避免了$O(L^2)$的全序列并行开销。
公式 (5)中的输出由两部分组成：第一项$\mathbf{Q}_{[t]}\mathbf{S}_{[t]}^\top$是”历史贡献”（前面所有块的信息通过$\mathbf{S}_{[t]}$传递），第二项$(\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^\top \odot \mathbf{M}_C)\mathbf{V}_{[t]}$是”块内贡献”（当前块内位置之间的注意力）。

数学推导详解：分块形式的推导

目标：从递推形式推导出分块并行公式。
起点：$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]}^0 + \sum_{i=1}^r \boldsymbol{v}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top}$
块间状态更新：令$r = C$（取块内最后一个状态）：

$$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]}^C = \mathbf{S}_{[t]}^0 + \sum_{i=1}^{C} \boldsymbol{v}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top}$$

用矩阵形式写出$\sum_{i=1}^C \boldsymbol{v}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top} = \mathbf{V}_{[t]}^\top \mathbf{K}_{[t]}$，得到公式 (4)。
维度：$\mathbf{V}_{[t]} \in \mathbb{R}^{C\times d}$，$\mathbf{K}_{[t]} \in \mathbb{R}^{C\times d}$，$\mathbf{V}_{[t]}^\top \mathbf{K}_{[t]} \in \mathbb{R}^{d\times d}$。

输出推导:

$$\boldsymbol{o}_{[t]}^r = \mathbf{S}_{[t]}^r \boldsymbol{q}_{[t]}^r = \left(\mathbf{S}_{[t]}^0 + \sum_{i=1}^r \boldsymbol{v}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top} \right) \boldsymbol{q}_{[t]}^r$$
$$= \underbrace{\mathbf{S}_{[t]}^0 \boldsymbol{q}_{[t]}^r}_{\text{历史贡献}} + \underbrace{\sum_{i=1}^r \boldsymbol{v}_{[t]}^i (\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{q}_{[t]}^r)}_{\text{块内贡献}}$$

将所有$r = 1, \ldots, C$堆叠为矩阵，块内贡献变成$(\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^\top \odot \mathbf{M}_C)\mathbf{V}_{[t]}$，其中$\mathbf{M}_C$是$C \times C$的因果掩码（下三角全1矩阵），确保位置$r$只能看到$i \le r$的位置。
复杂度分析：块内$\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^\top$：$O(C^2 d)$，共$L/C$块，总共$O(LCd)$。块间$\mathbf{V}_{[t]}^\top \mathbf{K}_{[t]}$：$O(Cd^2)$，共$L/C$块，总共$O(Ld^2)$。合计$O(LCd + Ld^2)$。

3.2 DeltaNet：具有Delta更新规则的线性Transformer

原文翻译

上述线性Transformer使用简单的线性递推：$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$。这可以看作是在每个时间步用新的键-值关联来加性更新记忆$\mathbf{S}_{t-1}$。然而，纯加性更新规则使得很难释放过去的键-值关联，当$L > d$时最终导致键”碰撞”，正如Schlag等人(2021)所指出的。理想情况下，模型应该学会删除不太重要的键-值关联以为新关联腾出空间，而且这种删除应该取决于新键与记忆内容之间的交互。
从快速权重编程 [Schmidhuber, 1992] 【快速权重编程是Schmidhuber提出的概念，将RNN的隐状态看作”快速权重”——一个可以在推理时快速更新的权重矩阵，用于从输入映射到输出。】 的角度来看，线性注意力的递推隐状态是将输入$\boldsymbol{q}_t$映射到输出$\boldsymbol{o}_t$的快速权重，使用类Hessian的更新规则，其记忆容量有限。另一方面，DeltaNet使用delta更新规则或Widrow-Hoff学习规则 [Widrow & Hoff, 1988] 【LMS（最小均方）算法，在信号处理中广泛应用。】 进行快速权重更新：$\mathbf{S}_t = \mathbf{S}_{t-1} - \beta_t(\mathbf{S}_{t-1}\boldsymbol{k}_t - \boldsymbol{v}_t)\boldsymbol{k}_t^\top$，其中$\beta_t$是学习率，$\mathbf{S}_{t-1}\boldsymbol{k}_t$代表当前预测，$\boldsymbol{v}_t$是目标值。该方法的名称源于基于”预测$\mathbf{S}_{t-1}\boldsymbol{k}_t$和目标$\boldsymbol{v}_t$之间的差（delta）”来更新权重的核心原则，这已被证明具有更好的记忆容量。这个过程也可以看作是用一步SGD来优化在线回归损失[a]：

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}\boldsymbol{k}_t - \boldsymbol{v}_t\|^2, \quad \mathbf{S}_t = \mathbf{S}_{t-1} - \beta_t \nabla_{\mathbf{S}_{t-1}} \mathcal{L}_t(\mathbf{S}_{t-1}) = \mathbf{S}_{t-1} - \beta_t(\mathbf{S}_{t-1}\boldsymbol{k}_t - \boldsymbol{v}_t)\boldsymbol{k}_t^\top, \quad (6)$$

相比之下，线性Transformer可以被看作是在优化在线线性（负内积）损失 $\mathcal{L}_t = -\langle \mathbf{S}\boldsymbol{k}_t, \boldsymbol{v}_t \rangle$。[b]
DeltaNet的另一种解释是从键-值检索的角度。它首先使用当前键检索旧值，$\boldsymbol{v}_t^{\text{old}} = \mathbf{S}_{t-1}\boldsymbol{k}_t$。然后通过在旧值和当前值$\boldsymbol{v}_t$之间插值获得新值$\boldsymbol{v}_t^{\text{new}}$，替换记忆中的$\boldsymbol{v}_t^{\text{old}}$：

$$\boldsymbol{v}_t^{\text{new}} = \beta_t \boldsymbol{v}_t + (1 - \beta_t)\, \boldsymbol{v}_t^{\text{old}}, \qquad \mathbf{S}_t = \mathbf{S}_{t-1} \underbrace{-\boldsymbol{v}_t^{\text{old}} \boldsymbol{k}_t^\top}_{\text{删除}} \underbrace{+\boldsymbol{v}_t^{\text{new}} \boldsymbol{k}_t^\top}_{\text{写入}} \quad (7)$$

这里$\beta_t = \sigma(\mathbf{W}_\beta \boldsymbol{x}_t) \in (0, 1)$是一个软”写入强度”：当$\beta_t = 1$时，旧值被完全删除

且$\boldsymbol{v}_t^{\text{new}} = \boldsymbol{v}_t$；当$\beta_t = 0$时，记忆保持不变，$\mathbf{S}_t = \mathbf{S}_{t-1}$。输出计算与普通线性注意力相同，即$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$。该递推形式的复杂度与普通线性注意力相同，即$\mathcal{O}(Ld^2)$。

Schlag等人(2021)证明DeltaNet在小规模语言建模和合成上下文检索任务上优于普通线性Transformer。然而，他们的训练算法——线性Transformer的内存高效递推实现的扩展——是严格串行的，因此硬件效率低，这促使我们推导出等价的分块算法来大规模训练DeltaNet。

[a]这个视角早在Widrow & Hoff (1988)中就被讨论了，后来被称为最小均方（LMS）算法，在信号处理中得到了广泛应用。

[b]虽然二次损失（DeltaNet使用的）提供的梯度与误差幅度成比例，在预测远离目标时提供更强的修正，但线性损失的梯度无论预测误差大小都保持不变。这种梯度行为为线性Transformer在上下文检索任务上不如DeltaNet提供了直观的解释。

通俗解释

DeltaNet的核心思想可以用”可擦写白板”来类比：

普通线性注意力（加性更新）：就像一块白板，你只能往上面写新内容，不能擦除。写满之后就会变得混乱，新旧信息混在一起——这就是”键碰撞”问题。

DeltaNet（delta规则更新）：像一块可以擦写的白板。每次要写新信息时，先查看白板上这个位置有没有旧内容（$\boldsymbol{v}_t^{\text{old}} = \mathbf{S}_{t-1}\boldsymbol{k}_t$），如果有就先擦掉（$-\boldsymbol{v}_t^{\text{old}}\boldsymbol{k}_t^\top$），然后写入新旧混合的内容（$+\boldsymbol{v}_t^{\text{new}}\boldsymbol{k}_t^\top$）。$\beta_t$控制”擦除力度”：$\beta_t = 1$时完全用新内容覆盖，$\beta_t = 0$时保持不变。

为什么DeltaNet检索能力更强：从在线学习的角度看，DeltaNet用的是均方误差损失（$\frac{1}{2}\|\mathbf{S}\boldsymbol{k}_t - \boldsymbol{v}_t\|^2$），梯度大小与预测误差成正比——预测越差修正越大。而线性注意力用的是线性损失，梯度恒定，不管预测好坏修正力度一样，所以学习效率低。

数学推导详解：DeltaNet的两种等价形式

形式1：在线SGD视角

损失函数：$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}\boldsymbol{k}_t - \boldsymbol{v}_t\|^2$

梯度计算：

$$\nabla_{\mathbf{S}}\mathcal{L}_t = (\mathbf{S}\boldsymbol{k}_t - \boldsymbol{v}_t)\boldsymbol{k}_t^\top$$

（这里$\mathbf{S}\boldsymbol{k}_t - \boldsymbol{v}_t \in \mathbb{R}^d$，$\boldsymbol{k}_t^\top \in \mathbb{R}^{1\times d}$，外积得到$\mathbb{R}^{d\times d}$矩阵）

一步SGD更新：$\mathbf{S}_t = \mathbf{S}_{t-1} - \beta_t(\mathbf{S}_{t-1}\boldsymbol{k}_t - \boldsymbol{v}_t)\boldsymbol{k}_t^\top$

形式2：检索-删除-写入视角

展开SGD更新：

$$\begin{aligned}\mathbf{S}_t &= \mathbf{S}_{t-1} - \beta_t\mathbf{S}_{t-1}\boldsymbol{k}_t\boldsymbol{k}_t^\top + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top \\ &= \mathbf{S}_{t-1} - \underbrace{(\mathbf{S}_{t-1}\boldsymbol{k}_t)}_{\boldsymbol{v}_t^{\text{old}}}\underbrace{(\beta_t\boldsymbol{k}_t^\top)}_{\text{缩放}} + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top\end{aligned}$$

重新整理，定义$\boldsymbol{v}_t^{\text{old}} = \mathbf{S}_{t-1}\boldsymbol{k}_t$和$\boldsymbol{v}_t^{\text{new}} = \beta_t\boldsymbol{v}_t + (1-\beta_t)\boldsymbol{v}_t^{\text{old}}$：

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \boldsymbol{v}_t^{\text{old}}\boldsymbol{k}_t^\top + \boldsymbol{v}_t^{\text{new}}\boldsymbol{k}_t^\top$$

验证：$\boldsymbol{v}_t^{\text{new}}\boldsymbol{k}_t^\top - \boldsymbol{v}_t^{\text{old}}\boldsymbol{k}_t^\top = (\beta_t\boldsymbol{v}_t + (1-\beta_t)\boldsymbol{v}_t^{\text{old}} - \boldsymbol{v}_t^{\text{old}})\boldsymbol{k}_t^\top = \beta_t(\boldsymbol{v}_t - \boldsymbol{v}_t^{\text{old}})\boldsymbol{k}_t^\top = \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top - \beta_t\mathbf{S}_{t-1}\boldsymbol{k}_t\boldsymbol{k}_t^\top$，与SGD形式一致。

数值例子：设$d=2$，$\mathbf{S}_0 = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$，$\boldsymbol{k}_1 = [1,0]^\top$，$\boldsymbol{v}_1 = [5,6]^\top$，$\beta_1 = 0.8$。

检索旧值：$\boldsymbol{v}_1^{\text{old}} = \mathbf{S}_0\boldsymbol{k}_1 = [1,0]^\top$

计算新值：$\boldsymbol{v}_1^{\text{new}} = 0.8\times[5,6]^\top + 0.2\times[1,0]^\top = [4.2,4.8]^\top$

更新：$\mathbf{S}_1 = \mathbf{S}_0 - [1,0]^\top[1,0] + [4.2,4.8]^\top[1,0] = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} - \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix} + \begin{bmatrix}4.2 & 0\\ 4.8 & 0\end{bmatrix} = \begin{bmatrix}4.2 & 0\\ 4.8 & 1\end{bmatrix}$

关键概念

DeltaNet vs. 线性注意力的本质区别：线性注意力的状态更新$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t\boldsymbol{k}_t^\top$是纯加性的，只能写入不能删除。DeltaNet的更新$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top) + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top$先通过乘以$(\mathbf{I} - \beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top)$选择性地”遗忘”旧信息（在$\boldsymbol{k}_t$方向上），然后写入新信息。这种”先擦后写”的机制赋予了DeltaNet更强的记忆管理能力和检索精度。

4 在序列维度上并行化DeltaNet

4.1 内存高效的重参数化

原文翻译

我们首先观察到$\mathbf{S}_t$可以表示为纯加性形式$\mathbf{S}_t = \sum_{i=1}^t \boldsymbol{u}_i\boldsymbol{k}_i^\top$（$\boldsymbol{u}_i, \boldsymbol{k}_i \in \mathbb{R}^d$），因为我们可以简单地令$\boldsymbol{u}_i = \boldsymbol{v}_i^{\text{new}} - \boldsymbol{v}_i^{\text{old}} = \beta_i(\boldsymbol{v}_i - \boldsymbol{v}_i^{\text{old}})$。回顾§3.1中简单线性注意力的形式$\mathbf{S}_t = \sum_{i=1}^t \boldsymbol{v}_i\boldsymbol{k}_i^\top$。因此，DeltaNet只是将线性注意力中的值向量$\boldsymbol{v}_i$替换为”伪”值向量$\boldsymbol{u}_i$。一旦构造了$\boldsymbol{u}_i$，其余计算可以像普通线性注意力一样进行，即$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\,\mathbf{U}$，其中$\mathbf{U} \in \mathbb{R}^{L\times d}$是$\boldsymbol{u}_i$向量的按行拼接。

然而，朴素地计算$\boldsymbol{u}_t$需要显式具现化$\mathbf{S}_{t-1}$来计算$\boldsymbol{v}_t^{\text{old}}$，这需要$\mathcal{O}(d^2)$的内存。我们现在展示可以在$\mathcal{O}(d)$内存下获得$\boldsymbol{u}_t$，无需显式具现化$\mathbf{S}_{t-1}$。我们的简单证明（通过归纳法）依赖于Householder矩阵乘积的WY表示 [Bischof & Van Loan, 1985] 【WY表示最初用于QR分解中高效计算Householder变换的乘积，这里被创造性地应用到RNN隐状态的高效表示。】 的应用。

基础情况很明确，因为$\mathbf{S}_1 = \beta_1\boldsymbol{v}_1\boldsymbol{k}_1^\top$，所以$\boldsymbol{u}_1 = \beta_1\boldsymbol{v}_1$。对于归纳步骤，我们首先观察DeltaNet更新由以下给出：

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \boldsymbol{v}_t^{\text{old}}\boldsymbol{k}_t^\top + \boldsymbol{v}_t^{\text{new}}\boldsymbol{k}_t^\top = \mathbf{S}_{t-1} - \beta_t\left(\mathbf{S}_{t-1}\boldsymbol{k}_t\right)\boldsymbol{k}_t^\top + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top) + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top,$$

这可以看作是对前一状态应用广义Householder变换（即与单位矩阵加秩一矩阵相

乘）。归纳步骤如下：

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top = \sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top + \underbrace{\beta_t \left(\boldsymbol{v}_t - \sum_{i=1}^{t-1} \boldsymbol{u}_i \left(\boldsymbol{k}_i^\top \boldsymbol{k}_t \right) \right)}_{\text{定义为 } \boldsymbol{u}_t} \boldsymbol{k}_t^\top = \sum_{i=1}^{t} \boldsymbol{u}_i \boldsymbol{k}_i^\top \tag{8}$$

注意$\boldsymbol{u}_t$不需要具现化任何隐状态，且只需要$\mathcal{O}(d)$内存来计算，从而完成了证明。虽然我们避免了具现化$\mathbf{S}_t$，但计算所有L个$\boldsymbol{u}_t$（即$\mathbf{U}$）需要$\mathcal{O}(L^2 d)$，而且不能完全并行化——不像线性注意力中可以在$\mathcal{O}(1)$步内并行计算所有值向量$\mathbf{V}$。我们现在展示上述技巧仍然可以实现DeltaNet的高效分块并行形式。

通俗解释

这一节的核心洞察非常精妙：

问题：DeltaNet的递推$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$不是纯加性的（因为有$\mathbf{S}_{t-1}$乘以矩阵的项），不能直接用线性注意力的分块并行方法。

关键发现：虽然DeltaNet的状态更新不是加性的，但状态本身可以表示为加性形式$\mathbf{S}_t = \sum_{i=1}^{t} \boldsymbol{u}_i \boldsymbol{k}_i^\top$！只要我们能计算出”伪值”$\boldsymbol{u}_i$，就可以复用线性注意力的所有并行技巧。

$\boldsymbol{u}_t$的计算：$\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \sum_{i=1}^{t-1} \boldsymbol{u}_i(\boldsymbol{k}_i^\top \boldsymbol{k}_t))$。这个公式只需要之前的$\boldsymbol{u}_i$向量（$d$维）和键向量之间的内积（标量），完全不需要具现化$d \times d$的隐状态矩阵！

Householder变换的联系：$\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$是一个广义Householder矩阵（单位矩阵加秩一扰动）。WY表示是数值线性代数中专门用来高效表示多个Householder矩阵乘积的方法——这正是DeltaNet多步递推的数学结构。

数学推导详解：WY表示的归纳证明

命题：$\mathbf{S}_t = \sum_{i=1}^{t} \boldsymbol{u}_i \boldsymbol{k}_i^\top$，其中$\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \sum_{i=1}^{t-1} \boldsymbol{u}_i(\boldsymbol{k}_i^\top \boldsymbol{k}_t))$。

基础情况（$t = 1$）：$\mathbf{S}_1 = \mathbf{S}_0(\mathbf{I} - \beta_1 \boldsymbol{k}_1 \boldsymbol{k}_1^\top) + \beta_1 \boldsymbol{v}_1 \boldsymbol{k}_1^\top$。假设$\mathbf{S}_0 = \mathbf{0}$，则$\mathbf{S}_1 = \beta_1 \boldsymbol{v}_1 \boldsymbol{k}_1^\top = \boldsymbol{u}_1 \boldsymbol{k}_1^\top$，其中$\boldsymbol{u}_1 = \beta_1 \boldsymbol{v}_1$。

归纳假设：假设$\mathbf{S}_{t-1} = \sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top$成立。

归纳步骤:

$$
\begin{aligned}
\mathbf{S}_t &= \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top \\
&= \left(\sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top\right)(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top \\
&= \sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top - \left(\sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top\right) \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top \\
&= \sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top - \sum_{i=1}^{t-1} \boldsymbol{u}_i \underbrace{(\boldsymbol{k}_i^\top \boldsymbol{k}_t)}_{\text{标量}} \beta_t \boldsymbol{k}_t^\top + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top \\
&= \sum_{i=1}^{t-1} \boldsymbol{u}_i \boldsymbol{k}_i^\top + \underbrace{\left[\beta_t \boldsymbol{v}_t - \beta_t \sum_{i=1}^{t-1} \boldsymbol{u}_i (\boldsymbol{k}_i^\top \boldsymbol{k}_t)\right]}_{\boldsymbol{u}_t} \boldsymbol{k}_t^\top \\
&= \sum_{i=1}^{t} \boldsymbol{u}_i \boldsymbol{k}_i^\top
\end{aligned}
$$

关键观察: $\boldsymbol{u}_t$只依赖于$\boldsymbol{v}_t$、$\boldsymbol{k}_t$、β_t以及之前的$\boldsymbol{u}_1, \ldots, \boldsymbol{u}_{t-1}$和$\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{t-1}$，全部都是$d$维向量。不需要存储或计算$d \times d$的矩阵$\mathbf{S}_{t-1}$!

内存分析: $\boldsymbol{u}_t$的计算需要$\sum_{i=1}^{t-1} \boldsymbol{u}_i(\boldsymbol{k}_i^\top \boldsymbol{k}_t)$，即$t-1$个$d$维向量的加权和，权重为标量$\boldsymbol{k}_i^\top \boldsymbol{k}_t$。每个$\boldsymbol{u}_t$的计算$O(td)$，所有$L$个$\boldsymbol{u}_t$总计$O(L^2 d)$，但只需$O(d)$额外内存。

注意事项

虽然$\boldsymbol{u}_t$的计算只需$O(d)$内存，但计算全部L个$\boldsymbol{u}_t$仍然需要$O(L^2 d)$时间且是串行的（$\boldsymbol{u}_t$依赖于$\boldsymbol{u}_1, \ldots, \boldsymbol{u}_{t-1}$）。这看似是一个缺点，但接下来的分块形式会解决这个问题：在每个块内（大小C），串行计算C个$\boldsymbol{u}$向量只需$O(C^2 d)$时间，而块之间可以并行。总复杂度变为$O(LCd + Ld^2)$。

4.2 DeltaNet的分块并行形式

原文翻译

为了推导分块并行形式，我们首先展开递推:

$$
\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top = \sum_{i=1}^{t} \beta_i (\boldsymbol{v}_i \boldsymbol{k}_i^\top) \left(\prod_{j=i+1}^{t} (\mathbf{I} - \beta_j \boldsymbol{k}_j \boldsymbol{k}_j^\top)\right). \tag{9}
$$

我们然后定义以下变量: $\mathbf{P}_i^j = \prod_{t=i}^{j}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) \in \mathbb{R}^{d \times d}$，$\mathbf{H}_i^j = \sum_{t=i}^{j} \beta_t (\boldsymbol{v}_t \boldsymbol{k}_t^\top) \mathbf{P}_{t+1}^j \in \mathbb{R}^{d \times d}$，其中当$i > j$时$\mathbf{P}_i^j = \mathbf{I}$。直觉上，$\mathbf{P}_i^j$是应用于$\mathbf{S}_i$以获得$\mathbf{S}_j$的”衰减因子”，$\mathbf{H}_i^j$表示从token i开始对$\mathbf{S}_j$的贡献。（因此$\mathbf{S}_t = \mathbf{H}_1^t$。）分块递推可以写为:

$$
\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]}^0 \mathbf{P}_{[t]}^r + \mathbf{H}_{[t]}^r \tag{10}
$$

技巧是使用类似于§4.1中描述的方法来高效表示$\mathbf{P}_{[t]}^r, \mathbf{H}_{[t]}^r \in \mathbb{R}^{d\times d}$矩阵，使这些矩阵可以用$\mathcal{O}(d)$内存存储：

$$\mathbf{P}_{[t]}^r = \mathbf{I} - \sum_{i=1}^{r} \boldsymbol{w}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top}, \qquad \mathbf{H}_{[t]}^r = \sum_{i=1}^{r} \boldsymbol{u}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top} \quad \in \mathbb{R}^{d\times d} \tag{11}$$

$$\boldsymbol{w}_{[t]}^r = \beta_{[t]}^r \left(\boldsymbol{k}_{[t]}^r - \sum_{i=1}^{r-1} \boldsymbol{w}_{[t]}^i (\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{k}_{[t]}^r) \right), \quad \boldsymbol{u}_{[t]}^r = \beta_{[t]}^r \left(\boldsymbol{v}_{[t]}^r - \sum_{i=1}^{r-1} \boldsymbol{u}_{[t]}^i (\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{k}_{[t]}^r) \right) \quad \in \mathbb{R}^d \tag{12}$$

基于公式 (10)，我们可以获得隐状态和输出的块级递推：

$$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]} + \left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}^\top \right)^\top \mathbf{K}_{[t]}, \tag{13}$$

$$\mathbf{O}_{[t]} = \mathbf{Q}_{[t]} \mathbf{S}_{[t]}^\top + (\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{M}) \left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}^\top \right) \tag{14}$$

其中$\square_{[t]} = \square_{[t]}^{1:C} \in \mathbb{R}^{C\times d}$，$\square \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{U}, \mathbf{W}\}$定义了由堆叠$\boldsymbol{q}_t, \boldsymbol{k}_t, \boldsymbol{v}_t, \boldsymbol{o}_t, \boldsymbol{u}_t, \boldsymbol{w}_t$向量形成的分块矩阵。

通俗解释

这是本文最核心的技术贡献。让我用简单的话解释发生了什么：

与线性注意力分块形式的对比：线性注意力的分块形式（公式 (4)–(5)）很简单，因为状态更新$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]} + \mathbf{V}_{[t]}^\top \mathbf{K}_{[t]}$是纯加性的。但DeltaNet的状态更新包含了”遗忘”（乘以$\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$），导致相邻块之间的状态转移更复杂。

分块形式的结构：公式 (10)说的是：块t第r步的状态 = 块初始状态 × 衰减因子 + 块内新增贡献。关键是衰减因子$\mathbf{P}_{[t]}^r$和块内贡献$\mathbf{H}_{[t]}^r$都是$d \times d$矩阵，但借助WY表示可以用$O(Cd)$的向量来紧凑地表示（$\boldsymbol{w}_{[t]}^1, \ldots, \boldsymbol{w}_{[t]}^C$和$\boldsymbol{u}_{[t]}^1, \ldots, \boldsymbol{u}_{[t]}^C$）。

最终的分块公式：公式 (13)–(14)与线性注意力的分块形式（公式 (4)–(5)）非常相似！唯一的区别是值矩阵$\mathbf{V}_{[t]}$被替换为$\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}^\top$。这意味着一旦计算了$\mathbf{U}$和$\mathbf{W}$，剩余的计算可以复用线性注意力的高效实现。

数学推导详解：从块内递推到矩阵公式

块内输出推导：从$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]}^0 \mathbf{P}_{[t]}^r + \mathbf{H}_{[t]}^r$开始。

用WY表示代入：

$$\begin{aligned}
\mathbf{S}_{[t]}^r &= \mathbf{S}_{[t]}^0 (\mathbf{I} - \sum_{i=1}^{r} \boldsymbol{w}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top}) + \sum_{i=1}^{r} \boldsymbol{u}_{[t]}^i \boldsymbol{k}_{[t]}^{i\top} \\
&= \mathbf{S}_{[t]}^0 + \sum_{i=1}^{r} (\boldsymbol{u}_{[t]}^i - \mathbf{S}_{[t]}^0 \boldsymbol{w}_{[t]}^i) \boldsymbol{k}_{[t]}^{i\top}
\end{aligned}$$

输出计算：

$$\boldsymbol{o}_{[t]}^r = \mathbf{S}_{[t]}^r \boldsymbol{q}_{[t]}^r = \mathbf{S}_{[t]}^0 \boldsymbol{q}_{[t]}^r + \sum_{i=1}^{r} (\boldsymbol{u}_{[t]}^i - \mathbf{S}_{[t]}^0 \boldsymbol{w}_{[t]}^i)(\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{q}_{[t]}^r)$$

堆叠所有$r = 1, \ldots, C$并定义$\tilde{\mathbf{U}}_{[t]} = \mathbf{U}_{[t]} - \mathbf{W}_{[t]}\mathbf{S}_{[t]}^\top$（”修正伪值”）：

$$\mathbf{O}_{[t]} = \mathbf{Q}_{[t]}\mathbf{S}_{[t]}^\top + (\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^\top \odot \mathbf{M}_C)\tilde{\mathbf{U}}_{[t]}$$

块间状态更新：令$r = C$：

$$\begin{aligned}\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]}^C &= \mathbf{S}_{[t]} + \sum_{i=1}^{C}(\boldsymbol{u}_{[t]}^i - \mathbf{S}_{[t]}\boldsymbol{w}_{[t]}^i)\boldsymbol{k}_{[t]}^{i\top} \\ &= \mathbf{S}_{[t]} + (\mathbf{U}_{[t]} - \mathbf{W}_{[t]}\mathbf{S}_{[t]}^\top)^\top \mathbf{K}_{[t]}\end{aligned}$$

原文翻译：实际考量

在上面，公式 (12)是完全递推的，因此不能直接使用tensor core。为了解决这个问题，我们进一步利用UT变换 [Joffrain et al., 2006; Domínguez & Orti, 2018] 【UT变换是将Householder变换乘积转化为矩阵运算的数值线性代数技术。】（详见附录推导）：

$$\mathbf{T}_{[t]} = \left(\mathbf{I} + \mathrm{tril}(\mathrm{diag}(\beta_{[t]})\mathbf{K}_{[t]}\mathbf{K}_{[t]}^\top, -1)\right)^{-1} \mathrm{diag}\left(\beta_{[t]}\right) \tag{15}$$

$$\mathbf{W}_{[t]} = \mathbf{T}_{[t]}\mathbf{K}_{[t]}, \qquad \mathbf{U}_{[t]} = \mathbf{T}_{[t]}\mathbf{V}_{[t]} \tag{16}$$

以将大部分操作转化为矩阵乘法。下三角矩阵的逆可以通过前代法高效求解。

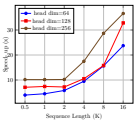
通俗解释

问题：$\boldsymbol{u}_t$和$\boldsymbol{w}_t$的递推计算（公式 (12)）是串行的，无法利用GPU的tensor core（专门用于矩阵乘法的硬件单元）。
UT变换的作用：将递推关系转化为一个矩阵方程。具体来说，$\mathbf{W}_{[t]} = \mathbf{T}_{[t]}\mathbf{K}_{[t]}$和$\mathbf{U}_{[t]} = \mathbf{T}_{[t]}\mathbf{V}_{[t]}$都是矩阵乘法，可以高效在GPU上执行。
关键步骤：计算$\mathbf{T}_{[t]}$需要求解一个$C \times C$下三角矩阵的逆。由于C很小（通常64或128），这在GPU上很快。$\mathrm{tril}(\cdot, -1)$表示取严格下三角部分（不含对角线），所以$\mathbf{I} + \mathrm{tril}(\ldots, -1)$是一个对角线为1的下三角矩阵，其逆总是存在且可以通过前代法$O(C^2)$计算。

原文翻译：速度比较

我们在Triton中实现了纯递推形式[a]和分块并行形式，并展示了不同序列长度(L)和头维度(d_{head})下的加速比（见右图），其中模型维度d为2048，我们改变批大小和序列长度使其乘积为16384。[b]我们的分块算法随着序列长度L和头维度d_{head}的增加实现了更大的加速，因为序列级并行性（高GPU占用率）和tensor core（快速矩阵乘法）变得更加重要。

[a]注意我们的递推核已经比Schlag等人(2021)的原始CUDA核快2倍。
[b]到目前为止，为了更容易说明，我们一直假设单头（$d_{\text{head}} = d$）。实际中我们使用多头，头维度d_{head}小于模型维度d。因此$\mathbf{S}_t \in \mathbb{R}^{d \times d_{\text{head}}}$。



r0.34 Speed-up of the chunkwise parallel form vs. the recurrent form.

Figure 1: 分块并行算法相对于纯递推算法的速度加速比。横轴为序列长度（千），纵轴为加速倍数。不同线代表不同的头维度d_{head}。

图 1解读

这张图展示了分块并行算法相对于纯递推算法的加速效果：
横轴：序列长度（以千为单位，从0.5K到16K）
纵轴：加速倍数（chunkwise相对于recurrent的速度比）
不同曲线：对应不同的头维度（$d_{\text{head}} = 64, 128, 256$）
关键趋势：(1) 序列越长，加速比越大——因为长序列下并行化的收益更显著；(2) 头维度越大，加速比越大——因为更大的矩阵乘法更能发挥tensor core的优势。在$L = 8K, d_{\text{head}} = 64$时达到约11.5倍加速，而$L = 2K, d_{\text{head}} = 256$时达到约13.7倍加速。
这验证了分块并行算法的实际效果。

Table 1: DeltaNet递推算法与分块并行算法在单块H100 GPU上的速度比较。

L	d_{head}	加速比
2048	64	5.5x
4096	64	7.6x
8192	64	11.5x
2048	128	8.9x
4096	128	13.2x
2048	256	13.7x

原文翻译：DeltaNet的完全并行形式

为完整起见，我们也讨论DeltaNet的完全并行形式。虽然我们使用了”伪”值的概念，但可以避免修改值。从公式 (9)，可以直接计算注意力矩阵$\mathbf{A}$：当$j \leq i$时$\mathbf{A}_{ij} = \boldsymbol{k}_j^\top \mathbf{P}_{j+1}^i \boldsymbol{q}_i$，否则为0。值得注意的是，$\mathbf{A}$有矩阵形式$\mathbf{A} = \left(\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M}\right)\mathbf{T}$，通过结合公式 (8)和(16)得到。然而，计算$\mathbf{T}$需要矩阵逆（公式 (15)），如果不进行进一步的算法改进，其复杂度随序列长度呈立方增长。因此，我们避免使用完全并行形式来训练DeltaNet；不过，这种形式推导出的”注意力”矩阵可能对RNN的可解释性研究有用。

通俗解释

完全并行形式意味着可以为DeltaNet定义一个类似于softmax注意力的注意力矩阵$\mathbf{A}$。然而由于需要矩阵逆（$O(L^3)$），在长序列上不实用。但这提供了一个有趣的理论联系：DeltaNet虽然是RNN，但有等价的注意力表示，可以像分析Transformer注意力模式那样分析DeltaNet的行为。

4.3 DeltaNet Transformer

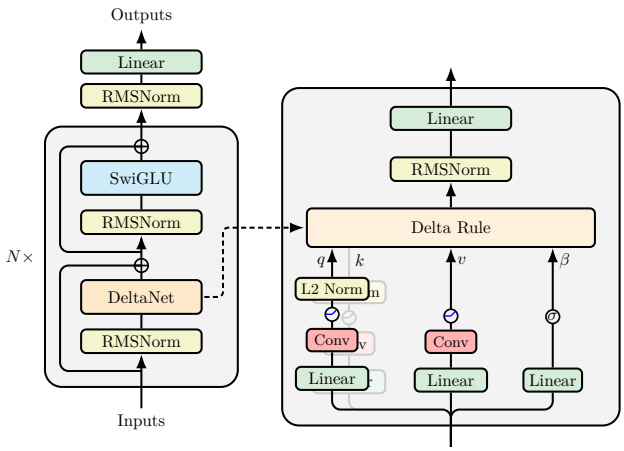


Figure 2: DeltaNet神经架构示意图。

原文翻译

我们描述如何使用DeltaNet层基本单元和标准模块来构建类Transformer模型。我们主要遵循LLaMA架构（Transformer++），简单地用DeltaNet层替换自注意力层。我们还在输出投影之前应用归一化以稳定训练。由于计算标量β_t项的额外参数可以忽略不计，参数分配与Transformer++大致相同，即DeltaNet层$4d^2$，SwiGLU FFN层$8d^2$。

图 2解读

这张架构图展示了DeltaNet的完整模型结构。从底向上：
(1) 输入$\boldsymbol{x}_t$先经过RMSNorm归一化（Pre-Norm架构）；
(2) 通过线性投影得到$\boldsymbol{q}_t, \boldsymbol{k}_t, \boldsymbol{v}_t$和$\beta_t$；
(3) 键和查询经过SiLU激活函数和L_2归一化；
(4) β_t通过sigmoid函数映射到$(0,1)$；
(5) DeltaNet核心：执行delta规则更新$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$，输出$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$；
(6) 输出经过GroupNorm归一化后做输出线性投影；
(7) 加上残差连接，再送入SwiGLU前馈网络（同样有残差连接）。
整体架构与LLaMA非常相似，只是把softmax自注意力换成了DeltaNet层。

原文翻译：特征映射和归一化

我们的键/查询向量由以下给出：$\boldsymbol{k}_t = \frac{\mathrm{SiLU}(\mathbf{W}_K \boldsymbol{x}_t)}{\|\mathrm{SiLU}(\mathbf{W}_K \boldsymbol{x}_t)\|_2}$，$\boldsymbol{q}_t = \frac{\mathrm{SiLU}(\mathbf{W}_Q \boldsymbol{x}_t)}{\|\mathrm{SiLU}(\mathbf{W}_Q \boldsymbol{x}_t)\|_2}$。Schlag等人(2021)最初使用”ELU+1”来非线性变换键/查询向量。我们改用SiLU激活函数，发现性能更好。
对于稳定性，确保转移矩阵每个特征值的模不超过1至关重要。$\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$的特征值为：重数$d-1$的1，和重数1的$1 - \beta_t \|\boldsymbol{k}_t\|_2$。Schlag等人(2021)使用$L_1$范数来归一化查询/键向量，确保$0 \le 1 - \beta_t \|\boldsymbol{k}_t\|_2 \le 1$。我们改用$L_2$归一化，发现性能更好，且提供了更直观的解释：当$\beta_t = 1$时，$\mathbf{I} - \boldsymbol{k}_t \boldsymbol{k}_t^\top$变成一个投影矩阵，在一个子空间中擦除信息，同时保留其

他$d-1$个子空间。这有利于在实现更有针对性的遗忘的同时保留信息。

通俗解释

为什么需要归一化键向量：转移矩阵$\mathbf{I}-\beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top$的特征值决定了信息传递的稳定性。如果某个特征值的绝对值大于1，每一步都会放大信号，导致数值爆炸。L_2归一化使得$\|\boldsymbol{k}_t\|_2=1$，此时特征值为$1-\beta_t$（一个）和1（其余$d-1$个）。因为$\beta_t\in(0,1)$，所以$1-\beta_t\in(0,1)$，所有特征值都≤ 1，保证稳定。

L_2归一化的几何意义：当$\|\boldsymbol{k}_t\|_2=1$且$\beta_t=1$时，$\mathbf{I}-\boldsymbol{k}_t\boldsymbol{k}_t^\top$是正交投影到$\boldsymbol{k}_t$的正交补空间上的投影矩阵。这意味着完全擦除$\boldsymbol{k}_t$方向的信息，保留所有其他方向——这是非常精确的”定向遗忘”。

4.4 混合模型

原文翻译

遵循最近将亚二次token混合层与现有神经网络原语结合的工作，我们也实验了DeltaNet模型的混合化。

卷积层。 最近的线性递推模型通常在查询/键/值投影后加入轻量级深度可分离卷积层。这种”短卷积”层推广了位移SSM，在参数数量和计算成本上都很高效。我们也在查询/键/值投影后添加了短卷积层。

局部滑动窗口和全局注意力。 线性注意力主要使用基于内容的寻址机制，缺乏位置信息。Arora等人(2024a)还认为线性注意力缺乏执行精确局部token位移和比较的能力，因此在检索密集任务上面临困难。基于此，我们实验了两种不同的混合架构：

(1) 滑动窗口注意力(SWA)：遵循Griffin和Samba，交替使用DeltaNet层和SWA层。

(2) 全局注意力：遵循H3的策略，仅替换两层为全局注意力：第二层和第$(\frac{N}{2}+1)$层（N是总层数）。

通俗解释

混合模型的直觉是”取各家之长”：

短卷积：捕捉局部模式（如n-gram），类似于在DeltaNet的”远距离记忆”之外加上”短期记忆”。

滑动窗口注意力：在固定大小的局部窗口内使用完整的softmax注意力，精确处理局部依赖关系（如相邻几个token之间的精确比较）。

全局注意力（仅2层）：在整个序列上使用完整注意力。虽然只有2层，但研究表明这足以提供DeltaNet所缺乏的全局精确检索能力。这是一种极具性价比的混合策略——只用2层$O(L^2)$注意力就能显著提升整体性能。

5 实验研究

5.1 合成基准测试

原文翻译

我们在三个合成基准上进行评估：多查询关联检索（MQAR），机械架构设计（MAD），和上下文语言学习（RegBench）。

5.1.1 MQAR（多查询关联检索）

原文翻译

MQAR评估语言模型在面对多个检索查询时从上下文中回忆信息的能力。我们使用Arora等人的训练设置，DeltaNet使用2个头。这些实验不使用卷积。图 3显示DeltaNet在最难设置下表现完美（即使没有卷积），并且在低维设置下优于Mamba（使用卷积）。

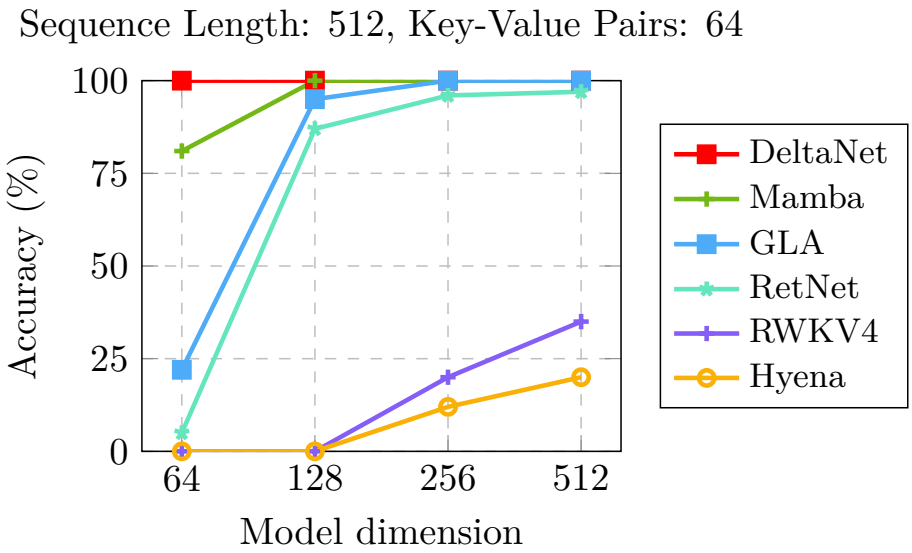


Figure 3: MQAR上的准确率(%)。不同子图对应不同的序列长度和键值对数量设置。横轴为模型维度，纵轴为准确率。

图 3解读

MQAR是一个合成任务：给定一系列键-值对，模型需要根据查询键检索对应的值。多个子图展示了不同难度级别：

从左到右/从上到下：随着序列长度增加和键-值对数量增加，任务变得更困难。

关键发现：(1) DeltaNet（绿色）在最难的设置下达到接近100%准确率，而其他模型（Mamba、GLA、RetNet等）性能下降明显。(2) DeltaNet甚至不需要卷积就能表现出色，而Mamba依赖卷积层。(3) 在低维设置下DeltaNet的优势尤为明显，说明delta规则在有限状态空间下的信息管理效率更高。

这验证了DeltaNet的核心优势：通过”先擦后写”的机制，DeltaNet能够更高效地管理有限的记忆空间，在关联检索任务上大幅领先。

5.1.2 MAD基准

Table 2: MAD合成基准上的结果。除DeltaNet外的结果直接来自Poli等人(2024)。(Multi-head) Hyena、DeltaNet和Mamba使用卷积，GLA不使用。

模型	Compress	Fuzzy Recall	In-Context Recall	Memorize	Noisy Recall	Selective Copy	平均
Transformer	51.6	29.8	94.1	85.2	86.8	99.6	**74.5**
Hyena	45.2	7.9	81.7	**89.5**	78.8	93.1	66.0
MH Hyena	44.8	14.4	99.0	89.4	98.6	93.0	73.2
Mamba	**52.7**	6.7	90.4	**89.5**	90.1	86.3	69.3
GLA	38.8	6.9	80.8	63.3	81.6	88.6	60.0
DeltaNet	42.2	**35.7**	**100**	52.8	**100**	**100**	71.8

表 2解读

MAD基准包含6个合成token操作任务。关键观察：

DeltaNet的强项：Fuzzy Recall（35.7%，远超其他模型）、In-Context Recall（100%）、Noisy Recall（100%）、Selective Copy（100%）。所有与”检索/回忆”相关的任务DeltaNet都是最强的，完美验证了delta规则对记忆管理的改进。

DeltaNet的弱项：Memorize任务只有52.8%（Transformer/Mamba约85–89%）。这说明DeltaNet虽然擅长”检索”，但在”记忆所有内容”方面不如拥有更大有效状态空间的模型。

总体排名：Transformer最强（74.5%），DeltaNet第二（71.8%），远超Mamba（69.3%）和GLA（60.0%）。

5.2 语言建模

原文翻译：实验设置

遵循先前工作，我们在Wikitext困惑度和零样本常识推理任务上进行评估，包括LAMBADA (LMB.)、PiQA、HellaSwag (Hella.)、WinoGrande (Wino.)、ARC-easy (ARC-e) 和 ARC-challenge (ARC-c)。遵循Arora等人(2024a)，我们还在真实世界的召回密集任务上评估模型，包括FDA、SWDE和SQuAD。超参数设置详见附录 A.1。

Table 3: 与Transformer++、RetNet、Mamba和GLA的主要语言建模结果对比。所有模型在SlimPajama数据集的相同子集上用Mistral分词器训练。混合模型中，”Sliding Attn”每隔一层交替使用滑动窗口注意力，”Global Attn”在两层使用完整全局注意力。340M/1.3B模型分别训练15B/100B token。最后一列表示循环状态大小相对于层数与模型维度乘积的扩展比。

模型	Wiki. ppl↓	LMB. ppl↓	LMB. acc↑	PIQA acc↑	Hella. acc_n↑	Wino. acc↑	ARC-e acc↑	ARC-c acc_n↑	Avg.	SWDE acc↑	SQuAD acc↑	FDA acc↑
340M params / 15B tokens												
Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2	42.2	22.1	21.4
RetNet	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0	13.3	27.6	2.9
Mamba (w. conv)	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8	12.4	23.0	2.1
GLA (w/o. conv)	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5	18.6	27.2	8.1
DeltaNet (w. conv)	**28.24**	**37.37**	**32.1**	64.8	34.3	52.2	45.8	23.5	**42.1**	26.4	28.9	12.8
+ Sliding Attn	27.06	38.17	33.4	64.0	35.3	50.9	45.9	23.2	42.1	39.3	32.5	18.8
+ Global Attn	27.51	35.04	33.5	64.0	34.5	51.7	46.0	23.3	42.1	42.9	32.1	23.1
1.3B params / 100B tokens												
Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9	66.6	31.5	27.4
RetNet	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9	42.8	34.7	14.3
Mamba (w. conv)	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0	41.4	35.2	6.2
GLA (w/o. conv)	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	51.0	50.6	42.6	19.9
DeltaNet (w. conv)	**16.87**	**12.21**	**48.9**	71.2	**50.2**	53.6	57.2	**28.3**	**51.6**	49.5	37.4	17.2
+ Sliding Attn	16.56	11.74	49.2	71.8	51.1	52.8	58.9	28.8	52.1	53.3	43.3	22.3
+ Global Attn	16.55	12.40	48.8	70.8	50.7	54.2	58.4	28.1	51.8	71.0	43.0	29.8
DeltaNet 消融实验 (340M)												
L_1-norm & 1+ELU	31.12	55.96	26.3	63.9	33.0	50.9	44.3	21.8	40.1	14.5	23.9	6.2
L_2-norm & 1+ELU	28.03	37.62	32.2	65.7	34.7	51.8	45.4	22.5	42.1	23.8	28.6	13.1
L_2-norm & ReLU	28.75	43.53	30.2	64.0	33.9	48.9	45.6	22.8	40.9	27.2	26.7	9.0

表 3解读

这是本文最重要的实验表格。关键发现：

1. DeltaNet vs. 线性模型基线：在340M和1.3B两个规模下，DeltaNet在困惑度和零样本任务平均分上都是最好的线性递归模型。1.3B DeltaNet的Wiki困惑度16.87，零样本平均51.6%，优于Mamba(17.06, 50.0%)和GLA(17.22, 51.0%)。

2. DeltaNet甚至接近Transformer++：1.3B DeltaNet的困惑度(16.87)几乎与Transformer++(16.85)持平！

3. 混合模型的威力：加入滑动窗口注意力或全局注意力后，DeltaNet混合模型(16.55–16.56)超越了Transformer++(16.85)。尤其是DeltaNet + Global Attn在SWDE上达到71.0%（Transformer++为66.6%），展现了强大的检索能力。

4. 召回密集任务的状态大小效应：在1.3B规模下，DeltaNet在SWDE/SQuAD/FDA上不如GLA(256x状态)，因为DeltaNet受限于128x状态扩展比。状态大小对检索任务至关重要。

5. 消融实验：L_2-norm比L_1-norm提升巨大（困惑度从31.12降到28.03）；SiLU激活比1+ELU和ReLU更好。

Table 4: 3B模型在选定基准上的零样本性能。Llama-3.2-3B和PowerLM-3B是Transformer模型，其余为循环模型。

模型	ARC	HellaSwag	OBQA	PIQA	WinoGr.	MMLU	Avg.
Llama-3.2-3B	59.1	73.6	43.4	77.5	69.2	54.1	62.8
PowerLM-3B	60.5	74.6	43.6	79.9	70.0	45.0	62.3
DeltaNet-3B	60.4	72.8	41.0	78.5	65.7	40.7	59.8
RecurrentGemma-2B	57.0	71.1	42.0	78.2	67.6	31.8	57.9
RWKV-6-3B	49.5	68.6	40.6	76.8	65.4	28.4	54.9
Mamba-2.7B	50.3	65.3	39.4	75.8	63.1	26.1	53.3

表 4解读

3B规模的结果（使用1T token训练）。DeltaNet-3B（59.8%）是所有循环模型中最强的，显著超过RecurrentGemma-2B（57.9%）、RWKV-6-3B（54.9%）和Mamba-2.7B（53.3%）。但仍略逊于Transformer模型（Llama-3.2-3B的62.8%和PowerLM-3B的62.3%），差距约2–3个百分点。值得注意的是这些模型的训练token数不完全相同，因此不是严格可比的。

5.2.1 训练吞吐量

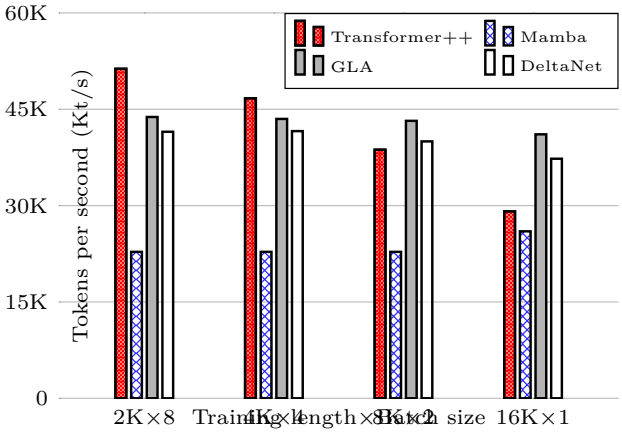


Figure 4: 1.3B模型在单块H100上的训练吞吐量比较。横轴为训练序列长度设置，纵轴为tokens/秒。

图 4解读

这张柱状图比较了不同1.3B模型在不同序列长度下的训练吞吐量（tokens/秒），在单块H100 GPU上测量。

关键发现：(1) DeltaNet的训练速度接近GLA，两者都显著快于Mamba；(2) 所有线性时间模型在长序列（8K, 16K）下都优于Transformer，因为Transformer的$O(L^2)$注意力复杂度在长序列下成为瓶颈；(3) DeltaNet + 滑动窗口注意力的混合模型速度略低于纯DeltaNet（因为要计算额外的局部注意力），但仍然快于Transformer。

这验证了分块并行算法的实际效率——DeltaNet不仅在质量上优于基线，在速度上也具有竞争力。

6 讨论与相关工作

6.1 DeltaNet vs. 状态空间模型/线性RNN

原文翻译

为了讨论DeltaNet与现有线性RNN（包括状态空间模型）的关系，我们首先引入一类具有矩阵值隐状态的通用关联RNN。给定矩阵值隐状态$\mathbf{S}_t \in \mathbb{R}^{d\times n}$和当前输入$\boldsymbol{x}_t \in \mathbb{R}^d$，这些模型具有以下形式：

$$\mathbf{S}_t = \mathbf{S}_{t-1} \bullet \mathbf{M}_t + \boldsymbol{v}_t \boldsymbol{k}_t^\top, \qquad \text{(递推)}$$
$$\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t, \qquad \text{(记忆读出)}$$

其中$\bullet$是关联算子（如Hadamard积、矩阵乘法等）。矩阵$\mathbf{M}_t$和向量$\boldsymbol{v}_t$、$\boldsymbol{k}_t$、$\boldsymbol{q}_t$是当前输入$\boldsymbol{x}_t$的（可能非线性的）函数。

Mamba和门控线性Transformer等近期模型使用廉价的逐元素递推更新，特别是Hadamard积：$\bullet = \odot$。而标准矩阵乘法$\mathbf{S}_{t-1}\mathbf{M}_t$可以建模超越逐元素递推的更丰富交互，但如果不对$\mathbf{M}_t$做结构假设，每次更新需要$O(dn^2)$，代价过高。

DeltaNet使用$\mathbf{M}_t = \mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$可以看作是利用结构化矩阵来高效建模超越逐元素递推的交互。我们的分块算法可以推广到更广泛的对角加低秩（DPLR）形式$\mathbf{M}_t = \mathbf{D} - \boldsymbol{a}_t \boldsymbol{b}_t^\top$的矩阵类。

关键概念

DeltaNet在模型谱中的位置：

逐元素递推（$\bullet = \odot$）$\longleftrightarrow$ 结构化矩阵乘法 $\longleftrightarrow$ 非结构化矩阵乘法

Mamba/GLA使用逐元素递推：$\mathbf{S}_t = \mathbf{S}_{t-1} \odot \mathbf{M}_t + \boldsymbol{v}_t \boldsymbol{k}_t^\top$。计算简单（$O(dn)$/步），但交互受限——隐状态矩阵的每个元素独立衰减。

DeltaNet使用结构化矩阵乘法：$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$。虽然$\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$是$d\times d$矩阵，但它是”单位矩阵 + 秩一”的特殊结构，仍可高效处理。

非结构化矩阵乘法：最具表达力但计算代价$O(d^2 n)$/步，不实用。

DeltaNet在计算效率和表达能力之间找到了一个优秀的平衡点。

6.2 局限性与未来工作

原文翻译

我们的工作有几个局限性。首先，在计算方面，虽然我们提出了新的硬件高效算法，但训练速度仍然落后于GLA。这是因为建模状态到状态依赖（如上所述）带来的开销，需要在核内”边际化”头维度，类似于softmax注意力的情况。然而，对于GLA，由于没有状态内依赖（一切都是逐元素的），因此很容易使用分块来支持任意大小的头维度。这个局限性可能限制DeltaNet的内存大小，从而降低召回密集任务的性能，正如我们在实验中观察到的。不过，可以采用块对角广义Householder转移矩阵，块大小适合GPU SRAM（如128），同时保持整体较大的头维度（从而较大的递推状态大小）。

我们还发现DeltaNet的长度泛化能力有限，而GLA和RetNet（以及在一定程度上Mamba）已被发现能够外推超出训练长度。我们推测这是因为DeltaNet缺少显式衰减因子。这可以通过在递推中加入门控项来改进，正如Gated DeltaNet所展示的。

通俗解释

两个主要局限：

1. 头维度受限：DeltaNet的$\boldsymbol{k}_t$向量参与矩阵乘法（$\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$），这限制了头维度不能太大（否则GPU SRAM装不下）。相比之下，GLA的逐元素操作不受此限制。解决方向：使用块对角结构，将大的头维度拆成多个小块。

2. 长度泛化差：DeltaNet训练在L长度的序列上，推理时用更长的序列效果不好。原因是DeltaNet没有”时间衰减”——如果β_t一直不为1，旧信息永远不会自然消失。GLA/RetNet有显式的衰减因子来”淡化”旧信息。后续工作Gated DeltaNet通过添加门控解决了这个问题。

6.3 相关工作

原文翻译

线性Transformer可以看作一种迭代Hopfield网络，这种联系可以提供关于线性注意力Transformer局限性和改进的视角。例如，vanilla线性Transformer使用类Hebbian的更新规则，已被证明具有有限的记忆容量。后来的Hopfield网络工作使用高阶多项式和指数核来增强记忆容量，这也与具有多项式核的线性注意力有关。另一方面，delta规则已被证明具有更好的记忆容量。从这个意义上说，在固定大小的递推状态下，使用delta规则能够在检索-记忆权衡曲线上达到更好的前沿。

尽管有这些优势，Irie等人(2023)揭示了delta更新规则在表达能力方面的理论局限性。DeltaNet的递推增强版本，如Recurrent DeltaNet和Modern Self-Referential Weight Matrix，以及mesa-layer，已被提出并被发现更优。然而，这些模型超越了线性RNN，不能在序列长度上并行化。这暗示了并行性和表达能力之间的基本权衡。如何在不牺牲并行性的情况下进一步增强DeltaNet仍是一个开放问题，TTT中使用的混合跨块非线性和块内线性策略可能提供合适的折中方案。

最后，delta规则与通过梯度下降进行的元学习或在线学习密切相关，这在近期的Longhorn和TTT中被重新审视。最近，Titans通过引入动量和权重衰减项改进了

TTT。

通俗解释

相关工作部分建立了丰富的理论联系：
Hopfield网络视角：线性注意力 $\leftrightarrow$ Hebbian学习（记忆容量有限）；DeltaNet $\leftrightarrow$ delta规则（记忆容量更强）；Softmax注意力 $\leftrightarrow$ 指数核Hopfield网络（记忆容量最强）。
并行性 vs 表达能力的权衡：更强的RNN变体（如Recurrent DeltaNet）有更强的表达能力，但无法并行化。DeltaNet是”可并行化”的最强线性RNN之一。TTT（Test-Time Training）通过块内用非线性更新、块间用线性传递来尝试突破这个权衡。
元学习视角：DeltaNet的更新规则本质上就是在线梯度下降——模型在推理时通过”学习”键-值关联来适应上下文。这个视角将DeltaNet与元学习、在线学习联系起来。

7 结论

原文翻译

我们描述了一种在序列长度维度上并行化DeltaNet训练的算法，在现代硬件上相比现有实现实现了显著的加速。这使得将DeltaNet扩展到中等规模的语言建模设置成为可能，在那里我们发现它与最近的线性递归基线相比表现良好。

8 致谢

原文翻译

本研究得到了MIT-IBM Watson AI Lab和MIT-Google Program for Computing Innovation的资金支持。我们感谢Mayank Mishra在训练和评估3B模型方面的协助，感谢Kazuki Irie对论文草稿的宝贵反馈，感谢Simran Arora、Liliang Ren和Eric Alcaide的深入讨论。我们也感谢Michael Poli和Armin Thomas分享MAD基准实验的原始结果，以及Fan Zhou指出了公式 (15)中的错误。

A 附录

A.1 超参数设置

原文翻译

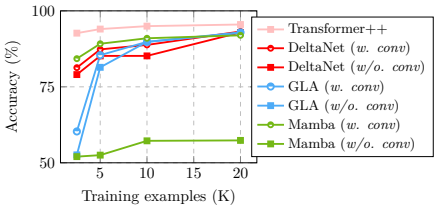
我们使用8块H100 GPU进行340M和1.3B语言建模实验。每个模型使用AdamW优化器，峰值学习率为3×10^{-4}。340M模型使用150亿token训练，批大小为50万token；1.3B模型使用1000亿token训练，批大小为200万token。我们使用余弦学习率调度，340M模型热身5亿token，1.3B模型热身10亿token。两种配置的初始和最终学习率设

为3×10^{-5}。权重衰减为0.01，梯度裁剪最大值为1.0。DeltaNet的头维度设为128，卷积层的核大小设为4。

A.2 合成任务：RegBench

原文翻译

我们在RegBench上进行了额外实验——一个旨在评估不同模型架构的上下文语言学习能力的合成数据集。该基准中的每个输入序列由10到20个字符串组成，这些字符串来自由概率有限自动机（PFA）定义的不同语言，因此模型需要即时从上下文中推断底层语言。测试时，模型在从保留PFA生成的测试序列上评估下一个token预测。这里我们再次发现DeltaNet与基线相比表现强劲。



r.4 Accuracy (%) on RegBench.

Figure 5: RegBench基准测试结果。展示了DeltaNet和基线模型在不同PFA（概率有限自动机）语言上的上下文学习性能。

图 5解读

RegBench测试模型的”上下文语言学习”能力——给定一些来自未知语言的样本字符串，模型需要推断出该语言的规则并预测下一个token。

关键发现：DeltaNet在大多数PFA语言上的表现优于或接近其他线性递归模型（Mamba、GLA、RetNet）。这再次验证了delta规则在上下文学习任务上的优势——它能够更有效地从上下文中”学习”和”更新”关联模式。

A.3 WY表示推导

原文翻译

为简化符号，我们只讨论第一个块。

我们首先通过归纳法证明$\mathbf{P}_n = \mathbf{I} - \sum_{t=1}^{n} \boldsymbol{w}_t \boldsymbol{k}_t^\top$：

$$
\begin{aligned}
\mathbf{P}_n &= \prod_{t=1}^{n} (\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) \\
&= \mathbf{P}_{n-1}(\mathbf{I} - \beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top) \\
&= (\mathbf{I} - \sum_{t=1}^{n-1} \boldsymbol{w}_t \boldsymbol{k}_t^\top)(\mathbf{I} - \beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top) \\
&= \mathbf{I} - \sum_{t=1}^{n-1} \boldsymbol{w}_t \boldsymbol{k}_t^\top - \beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top + (\sum_{t=1}^{n-1} \boldsymbol{w}_t \boldsymbol{k}_t^\top) \beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top \\
&= \mathbf{I} - \sum_{t=1}^{n-1} \boldsymbol{w}_t \boldsymbol{k}_t^\top - \underbrace{\left(\beta_n \boldsymbol{k}_n - \beta_n \sum_{t=1}^{n-1} \left(\boldsymbol{w}_t (\boldsymbol{k}_t^\top \boldsymbol{k}_n) \right) \right)}_{\boldsymbol{w}_n} \boldsymbol{k}_n^\top \\
&= \mathbf{I} - \sum_{t=1}^{n} \boldsymbol{w}_t \boldsymbol{k}_t^\top
\end{aligned}
$$

类似地，证明$\mathbf{S}_n = \sum_{t=1}^{n} \boldsymbol{u}_t \boldsymbol{k}_t^\top$（归纳法略，与主文中的推导等价）。

通俗解释

这是主文中WY表示的完整数学证明。关键步骤是将矩阵乘积$\prod_{t=1}^{n}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)$展开，利用归纳假设将其表示为$\mathbf{I} - \sum_{t=1}^{n} \boldsymbol{w}_t \boldsymbol{k}_t^\top$的形式。核心技巧是：乘以$(\mathbf{I} - \beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top)$时，前面的每个$\boldsymbol{w}_t \boldsymbol{k}_t^\top$项与$\beta_n \boldsymbol{k}_n \boldsymbol{k}_n^\top$相互作用，产生的结果可以被收集成一个新的$\boldsymbol{w}_n \boldsymbol{k}_n^\top$项。这正是WY表示的精髓——$n$个Householder矩阵的乘积可以紧凑地表示为$\mathbf{I} - \mathbf{W}\mathbf{K}^\top$，其中$\mathbf{W}, \mathbf{K} \in \mathbb{R}^{d \times n}$。

A.4 UT变换推导

原文翻译

这里提供§4.2中使用的UT变换的详细推导。具体来说，我们展示使用UT变换的矩阵公式（主文中的公式(15)和(16)）等价于递推更新方程（公式(12)）。
从$\boldsymbol{w}_{[t]}^r$的递推公式出发，定义$\mathbf{B}_{[t]} = \mathrm{diag}(\beta_{[t]})$和$\mathbf{L}_{[t]} = \mathrm{tril}(\mathbf{B}_{[t]}\mathbf{K}_{[t]}\mathbf{K}_{[t]}^\top, -1)$。
递推系统可以写为矩阵形式：$\mathbf{W}_{[t]} + \mathbf{L}_{[t]}\mathbf{W}_{[t]} = \mathbf{B}_{[t]}\mathbf{K}_{[t]}$
因此：$\mathbf{W}_{[t]} = (\mathbf{I} + \mathbf{L}_{[t]})^{-1}\mathbf{B}_{[t]}\mathbf{K}_{[t]} = \mathbf{T}_{[t]}\mathbf{K}_{[t]}$
其中$\mathbf{T}_{[t]} = (\mathbf{I} + \mathbf{L}_{[t]})^{-1}\mathbf{B}_{[t]}$。
$\mathbf{U}_{[t]}$的推导只需将最后一步的$\mathbf{K}_{[t]}$替换为$\mathbf{V}_{[t]}$，得到$\mathbf{U}_{[t]} = \mathbf{T}_{[t]}\mathbf{V}_{[t]}$。

数学推导详解：UT变换将递推转化为矩阵方程

目标：证明递推$\boldsymbol{w}_{[t]}^r = \beta_{[t]}^r(\boldsymbol{k}_{[t]}^r - \sum_{i=1}^{r-1} \boldsymbol{w}_{[t]}^i(\boldsymbol{k}_{[t]}^{i\top}\boldsymbol{k}_{[t]}^r))$等价于$\mathbf{W}_{[t]} = \mathbf{T}_{[t]}\mathbf{K}_{[t]}$。

展开为线性方程组：将第r行写出：

$$\boldsymbol{w}_{[t]}^r + \beta_{[t]}^r \sum_{i=1}^{r-1} \boldsymbol{w}_{[t]}^i (\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{k}_{[t]}^r) = \beta_{[t]}^r \boldsymbol{k}_{[t]}^r$$

这是关于$\boldsymbol{w}_{[t]}^1, \ldots, \boldsymbol{w}_{[t]}^C$的线性方程组。矩阵形式：

$$(\mathbf{I} + \mathbf{L}_{[t]})\mathbf{W}_{[t]} = \mathbf{B}_{[t]}\mathbf{K}_{[t]}$$

其中$\mathbf{L}_{[t]}$的第(r, i)元素（$r > i$）为$\beta_{[t]}^r (\boldsymbol{k}_{[t]}^{i\top} \boldsymbol{k}_{[t]}^r)$——正好是$\mathrm{tril}(\mathbf{B}_{[t]}\mathbf{K}_{[t]}\mathbf{K}_{[t]}^\top, -1)$的对应元素。
由于$\mathbf{I} + \mathbf{L}_{[t]}$是对角线全为1的下三角矩阵，其逆总是存在的，可以用前代法$O(C^2)$高效求解。

A.5 伪代码

```
1  def chunk_delta_rule_forward(Q, K, V, beta, C):
2      '''
3      Q/K/V: query, key, value of shape [L, d]
4      beta: beta of shape [L]
5      C: chunk size
6      '''
7      # L: sequence length, d: head dimension
8      L, d = Q.shape
9      # chunking
10     Q, K, V = map(lambda x: x.reshape(-1,C,d), [Q, K, V])
11     beta = beta.reshape(-1, C)
12     K_beta = K * beta.unsqueeze(-1)
13     V_beta = V * beta.unsqueeze(-1)
14
15     # compute eq. 10 with vectorized forward substitution for fast inverse
16     T = -(K_beta @ K.t()).tril(-1)
17     for i in range(1, C):
18         T[i, :i] = T[i, :i] + (T[i, :, None] * T[:, :i]).sum(-2)
19     T += torch.eye(C)
20     # compute Eq. 11
21     W = T @ K_beta
22     U = T @ V_beta
23     # chunkwise parallel. Eq. 8-9
24     S = torch.zeros(d, d)
25     O = torch.empty_like(V)
26     for i in range(L//C):
27         q_i, k_i, w_i = Q[i], K[i], W[i]
28         u_i = U[i] - w_i @ S
29         o_inter = q_i @ S
30         A_i = (q_i @ k_i.t()).tril()
31         o_intra = A_i @ u_i
32         S += k_i.t() @ u_i
33         O[i] = o_intra + o_inter
34     return O.reshape(L, d)
```

Figure 6: DeltaNet分块算法前向传播的PyTorch风格伪代码。为清晰起见，省略了批大小和注意力头数量维度。

伪代码逐行解读

第8–13行：数据准备

将长度L的序列切成L/C个大小为C的块。`K_beta`和`V_beta`是β缩放后的键和值。

第15–18行：计算UT变换矩阵T

`T = -(K_beta @ K.t()).tril(-1)` 计算$-\mathbf{L}_{[t]}$（严格下三角部分）。

`for`循环实现向量化的前代法来计算$(\mathbf{I}+\mathbf{L})^{-1}$——这是$O(C^2)$的操作，因为$C$很小（64或128），所以很快。

`T += torch.eye(C)`加上单位矩阵完成$\mathbf{T}$的构造。

第20–21行：计算WY表示的W和U

$\mathbf{W}=\mathbf{T}\cdot\mathbf{K}_\beta$：公式 (16)的第一部分。

$\mathbf{U}=\mathbf{T}\cdot\mathbf{V}_\beta$：公式 (16)的第二部分。这两步都是标准矩阵乘法，可以高效利用tensor core。

第23–31行：分块并行主循环

`u_i = U[i] - w_i @ S`：计算修正伪值$\tilde{\mathbf{U}}_{[t]}=\mathbf{U}_{[t]}-\mathbf{W}_{[t]}\mathbf{S}_{[t]}^\top$。数据来源：$\mathbf{U},\mathbf{W}$来自第20–21行（已在HBM中），$\mathbf{S}$是跨块传递的隐状态（驻留SRAM）。

`o_inter = q_i @ S`：历史贡献，$\mathbf{Q}_{[t]}\mathbf{S}_{[t]}^\top$。

`A_i = (q_i @ k_i.t()).tril()`：块内注意力矩阵$\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^\top\odot\mathbf{M}_C$。

`o_intra = A_i @ u_i`：块内贡献。

`S += k_i.t() @ u_i`：更新隐状态，$\mathbf{S}_{[t+1]}=\mathbf{S}_{[t]}+\mathbf{K}_{[t]}^\top\tilde{\mathbf{U}}_{[t]}$。

`O[i] = o_intra + o_inter`：合并两部分输出。

设计动机：第23–31行的结构与FlashLinearAttention几乎相同，唯一区别是值矩阵从$\mathbf{V}$变成了$\mathbf{U}-\mathbf{W}\mathbf{S}^\top$。这意味着可以最大程度地复用FlashLinearAttention的优化内核。

B 参考文献一览表

以下表格汇总了本文引用的主要文献。

Table 5: 参考文献一览表

文献	主要内容	在本文的引用原因
Vaswani et al. (2017), Transformer	提出自注意力机制和Transformer架构	本文改进的基础架构
Katharopoulos et al. (2020), Linear Transformer	提出线性注意力，将注意力复杂度降为线性	DeltaNet的前身，本文的起点
Schlag et al. (2021), DeltaNet	提出用delta规则替代线性注意力的加性更新	本文的核心模型，提出并行训练算法
Bischof & Van Loan (1985), WY表示	Householder矩阵乘积的紧凑表示	本文算法的数学基础
Gu & Dao (2023), Mamba	选择性状态空间模型，数据依赖的SSM	主要基线模型
Yang et al. (2023), GLA	门控线性注意力，数据依赖衰减	主要基线模型；FlashLinearAttention实现
Sun et al. (2023), RetNet	保留网络，固定指数衰减	基线模型
Dao (2022, 2023), FlashAttention 1&2	硬件感知的注意力加速算法	激发本文的硬件高效算法设计

文献	主要内容	在本文的引用原因
Arora et al. (2024a)	分析线性注意力的检索-吞吐量权衡	揭示线性注意力检索能力不足的问题
Widrow & Hoff (1988)	提出Widrow-Hoff学习规则/LMS算法	delta规则的原始来源
Schmidhuber (1992)	快速权重编程	线性注意力的隐状态视为快速权重
Hua et al. (2022)	提出分块线性注意力的并行算法	本文算法的基础框架
De et al. (2024), Griffin	混合线性递归+局部注意力架构	混合架构的参考
Touvron et al. (2023), LLaMA	LLaMA/Transformer++架构	DeltaNet Transformer的架构基础
Beck et al. (2024), xLSTM	扩展LSTM，含mLSTM变体	表格中的模型比较
Peng et al. (2024), RWKV-6	RWKV第六版，门控线性RNN	表格中的模型比较
Qin et al. (2024), HGRN2	层级门控递归网络第二版	表格中的模型比较
Dao & Gu (2024), Mamba-2	建立SSM与结构化矩阵的对偶性	统一框架的讨论
Poli et al. (2024), MAD	机械架构设计基准	合成基准实验
Arora et al. (2023a), Zoology	MQAR基准	合成基准实验
Akyürek et al. (2024), Reg-Bench	上下文语言学习基准	合成基准实验
Sun et al. (2024), TTT	测试时训练，推理时通过梯度下降学习	相关工作讨论
Yang et al. (2024), Gated DeltaNet	带门控的DeltaNet	解决长度泛化的后续工作
Behrouz et al. (2024), Titans	通过动量和衰减改进TTT	相关工作讨论
Joffrain et al. (2006), UT Transform	Householder乘积的UT变换	本文算法中的矩阵化技巧
Lewis et al. (2021), RAG	检索增强生成	说明检索能力的实用价值

B.1 线性递归模型总览

Table 6: 近期线性递归模型总览（按大致时间顺序）。这些工作使用矩阵值隐状态$\mathbf{S}_t \in \mathbb{R}^{d\times n}$，通过关联递推加外积更新。$\odot$为Hadamard积。带下标$t$的变量是输入$\boldsymbol{x}_t$的函数，不带下标的是学习参数或固定参数。

模型	递推	记忆读出
Linear Attention	$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$
+ Kernel	$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t\phi(\boldsymbol{k}_t)^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\phi(\boldsymbol{q}_t)$
+ Normalization	$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t\phi(\boldsymbol{k}_t)^\top$, $\boldsymbol{z}_t = \boldsymbol{z}_{t-1} + \phi(\boldsymbol{k}_t)$	$\boldsymbol{o}_t = \mathbf{S}_t\phi(\boldsymbol{q}_t)/(\boldsymbol{z}_t^\top\phi(\boldsymbol{q}_t))$
DeltaNet	$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top) + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$
RetNet	$\mathbf{S}_t = \gamma\mathbf{S}_{t-1} + \boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$
GLA	$\mathbf{S}_t = \mathbf{S}_{t-1} \odot (\mathbf{1}\boldsymbol{\alpha}_t^\top) + \boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$
Mamba	$\mathbf{S}_t = \mathbf{S}_{t-1} \odot \exp(-(\boldsymbol{\alpha}_t\mathbf{1}^\top) \odot \exp(\mathbf{A})) + (\boldsymbol{\alpha}_t \odot \boldsymbol{v}_t)\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t + \boldsymbol{d} \odot \boldsymbol{v}_t$
mLSTM	$\mathbf{S}_t = f_t\mathbf{S}_{t-1} + i_t\boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t / \max\{1, \lvert\boldsymbol{z}_t^\top\boldsymbol{q}_t\rvert\}$
Mamba-2	$\mathbf{S}_t = \gamma_t\mathbf{S}_{t-1} + \boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$
Gated DeltaNet	$\mathbf{S}_t = \mathbf{S}_{t-1}(\alpha_t(\mathbf{I} - \beta_t\boldsymbol{k}_t\boldsymbol{k}_t^\top)) + \beta_t\boldsymbol{v}_t\boldsymbol{k}_t^\top$	$\boldsymbol{o}_t = \mathbf{S}_t\boldsymbol{q}_t$